

Imperative and declarative approaches

- **Imperative**

- Step by step instructions
- Example of imperatively creating a nginx deployment with two replicas:

```
kubectl create deployment nginx-imperative --image nginx --replicas 2
```

- **Declarative**

- Used to define the desired/final state of the resource via manifests like YAML or JSON
- Example of declaratively creating a nginx deployment with two replicas:

#YAML file

```
kind: Deployment
metadata:
  labels:
    app: nginx-declarative
  name: nginx-declarative
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx-declarative
  template:
    metadata:
      labels:
        app: nginx-declarative
    spec:
      containers:
        - image: nginx
          name: nginx
```

#apply the YAML file

```
kubectl apply -f nginx-declarative.yaml
```

- Declarative method is often preferred since it is more scalable, repeatable and manageable, but imperative method is also helpful for quick interactions
- We can use the imperative approach to generate the yaml file with "--dry-run" and "-o yaml" parameters which just provides the YAML file without creating or applying it